

FIAP – Pós-Tech
Tech Challenge – Fase 1

Alunos:

Eduardo Coimbra Mattos
Guilherme dos Santos Condini
Paulo Henrique de Castilho
Ramon Cintas Gomes

Curso: IA para DEVs

SUMÁRIO

1. Introdução
2. Objetivo do projeto
3. Problema escolhido e dataset utilizado
4. Exploração dos dados
5. Estratégias de pré-processamento
6. Análise de correlação
7. Modelos utilizados e justificativa
8. Separação entre treino, validação e teste
9. Critério de escolha do melhor modelo
10. Treinamento e avaliação
11. Resultados e interpretação dos dados
12. Discussão crítica dos resultados
13. Possibilidade de uso prático
14. Conclusão
15. Link do repositório Git

1. Introdução

Este trabalho foi desenvolvido no contexto do Tech Challenge da Fase 1, cujo objetivo é construir uma solução inicial com foco em Inteligência Artificial para processamento de dados e apoio ao diagnóstico por meio de técnicas de Machine Learning.

Para atender a essa proposta, foi implementado um pipeline completo em Python para um problema de classificação binária em dados tabulares. O sistema realiza carregamento do dataset, validação da estrutura da base, limpeza e padronização dos dados, exploração estatística, análise de correlação, pré-processamento, treinamento de múltiplos modelos de classificação, comparação de desempenho por métricas, interpretação do modelo e geração automática de artefatos para documentação e entrega.

Embora o material de referência apresente exemplos na área de saúde humana, a base efetivamente utilizada neste projeto possui características de triagem clínica/veterinária, com variáveis relacionadas ao tipo de animal e sintomas observados. Dessa forma, a solução desenvolvida deve ser compreendida como uma prova de conceito para apoio à triagem, e não como um sistema autônomo de decisão, mantendo o profissional responsável como autoridade final no diagnóstico.

2. Objetivo do projeto

O objetivo principal deste projeto foi desenvolver um modelo preditivo de classificação binária capaz de identificar se um caso deve ser considerado perigoso ou não perigoso a partir das informações disponíveis na base de dados. No pipeline implementado, essa tarefa foi formulada a partir da coluna Dangerous, que deu origem à variável binária de saída dangerous_binary.

Além da predição, o projeto também teve como meta atender aos requisitos técnicos do desafio: exploração do dataset, tratamento de valores ausentes e inconsistências, construção de pipeline de pré-processamento em Python, análise de correlação, separação clara entre treino, validação e teste, uso de múltiplas técnicas de Machine Learning, avaliação com métricas adequadas e interpretação dos resultados com métodos de explicabilidade.

3. Problema escolhido e dataset utilizado

A tarefa foi tratada como um problema de classificação supervisionada binária. O sistema recebe como entrada o nome do animal (AnimalName) e cinco colunas de sintomas (symptoms1 a symptoms5) e produz como saída uma predição indicando se o registro deve ser classificado como perigoso (Yes) ou não perigoso (No).

O script utilizado no projeto valida explicitamente a presença das colunas AnimalName, symptoms1, symptoms2, symptoms3, symptoms4, symptoms5 e Dangerous, garantindo que o dataset esteja compatível com a estrutura

esperada para execução do pipeline. Essa validação ajuda a evitar erros de processamento e aumenta a confiabilidade do fluxo.

Do ponto de vista quantitativo, o resumo da base indica 869 linhas brutas com 30 duplicatas removidas, resultando em 839 registros limpos. A variável-alvo apresenta 819 casos yes e 20 casos no, evidenciando forte desbalanceamento em direção à classe perigosa.

A escolha desse dataset foi adequada ao objetivo do trabalho porque ele representa um cenário de triagem baseado em atributos predominantemente categóricos, exigindo transformações apropriadas antes da modelagem. Além disso, a base permite discutir decisões importantes de pré-processamento, como padronização textual, tratamento de redundâncias, codificação categórica e criação de atributos derivados.

4. Exploração dos dados

A etapa de exploração dos dados foi realizada com o objetivo de compreender a estrutura da base, verificar a distribuição da variável-alvo, identificar as categorias mais frequentes e levantar possíveis problemas de qualidade antes do treinamento dos modelos. O desafio exige explicitamente a análise estatística e a visualização de distribuições relevantes, e essa etapa foi pensada para cumprir esse requisito de forma organizada.

O resumo do dataset mostra a distribuição dos animais mais frequentes, com destaque para buffalo, sheep, bird, pig e elephant, que concentram grande parte dos registros. Também são listados os sintomas mais recorrentes, como fever, weight loss, decreased appetite, pain e diarrhoea, entre outros.

Com base nesses artefatos, o pipeline foi configurado para gerar automaticamente gráficos como:

- Distribuição da variável-alvo Dangerous, apresentando a quantidade de registros perigosos e não perigosos.
- Frequência dos animais mais recorrentes no dataset, permitindo observar a composição da base por categoria principal.
- Sintomas mais frequentes na base de dados após a padronização textual das colunas categóricas.
- Distribuição das features numéricas derivadas (unique_symptom_count, unknown_symptom_count, symptom_text_length) por classe da variável-alvo.

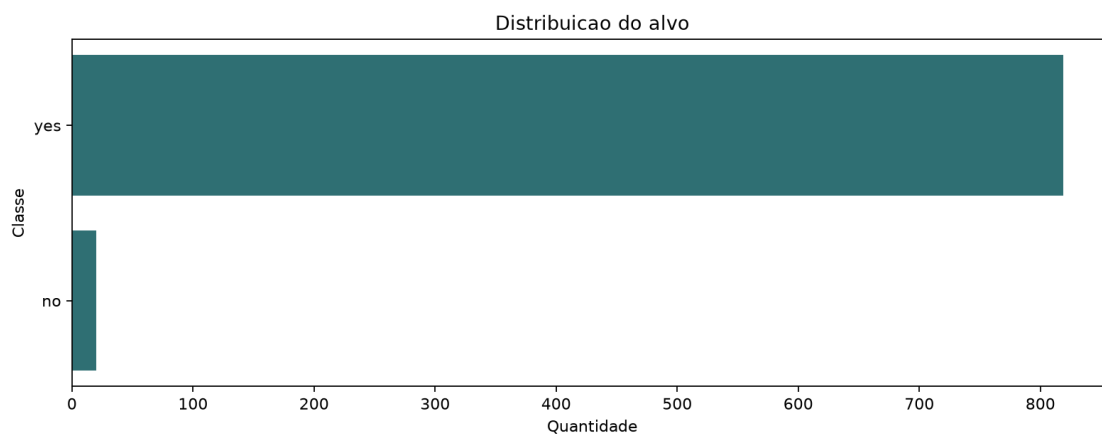


Figura 1 – Distribuição da variável-alvo Dangerous, mostrando a quantidade de registros perigosos e não perigosos.

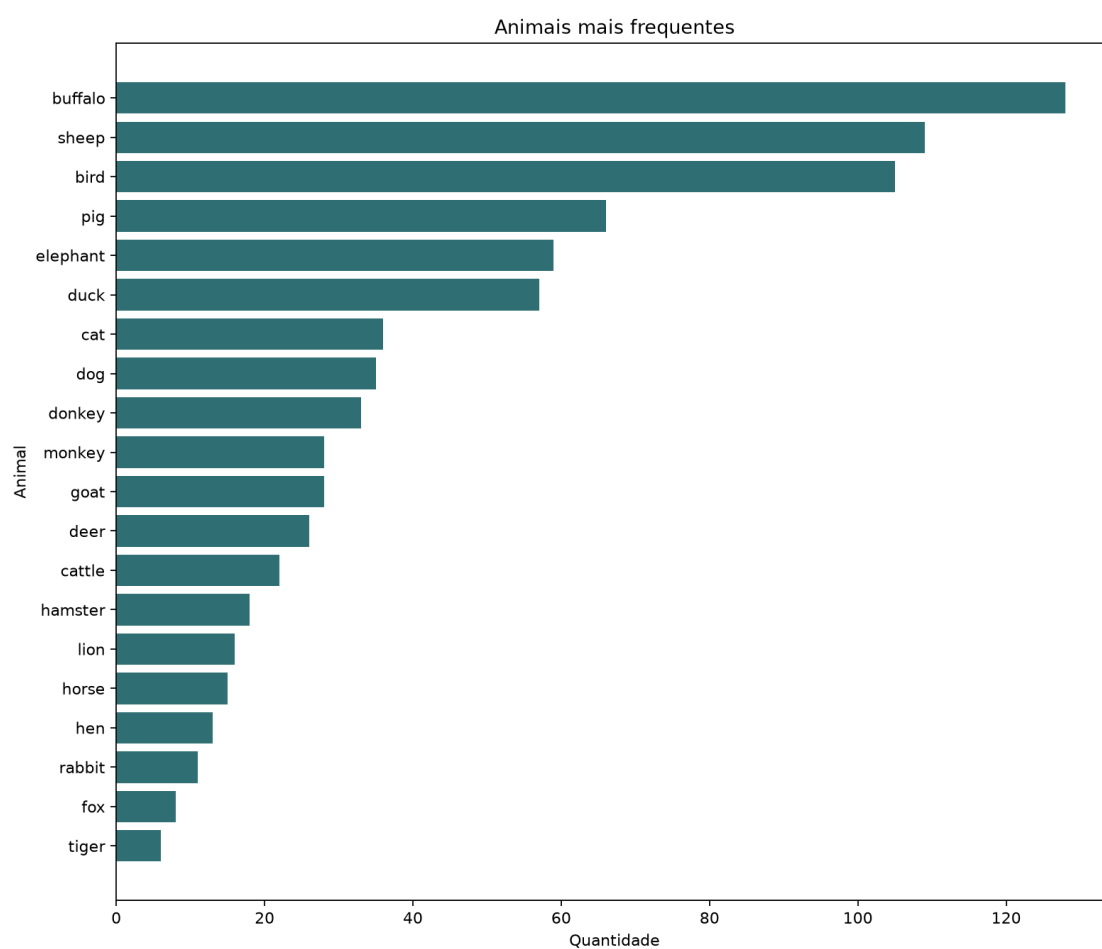


Figura 2 – Frequência dos animais mais recorrentes no dataset.

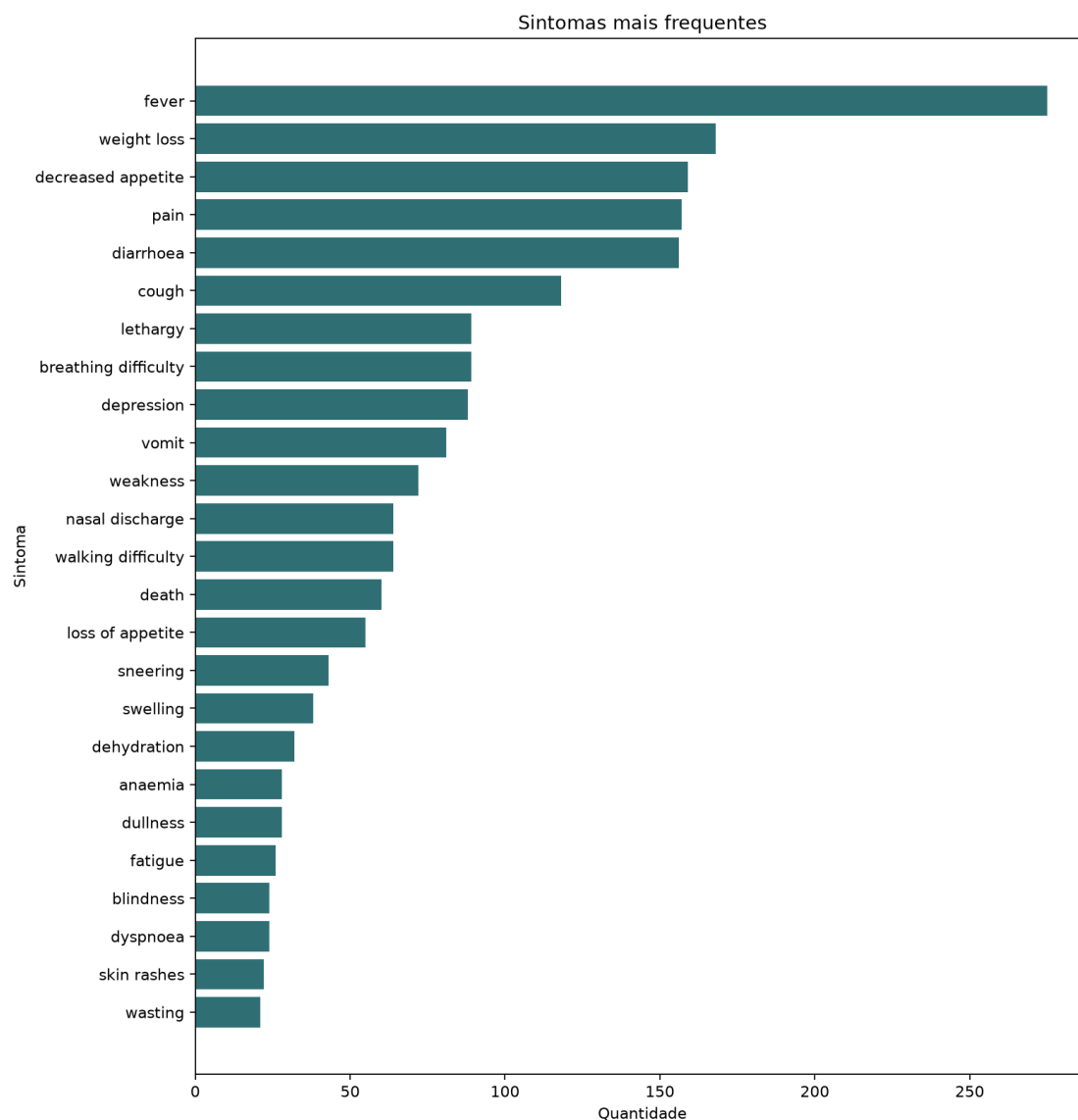


Figura 3 – Sintomas mais frequentes na base de dados após padronização textual.

A análise exploratória é importante porque antecipa possíveis desafios da modelagem. Distribuições muito assimétricas, categorias raras ou padrões específicos podem influenciar diretamente o desempenho dos algoritmos e justificar escolhas metodológicas como uso de codificação robusta, criação de variáveis auxiliares e priorização de métricas específicas.

5. Estratégias de pré-processamento

A etapa de pré-processamento foi desenvolvida para transformar os dados brutos em uma representação adequada para os algoritmos de classificação. O desafio pede explicitamente limpeza dos dados, tratamento de valores ausentes, conversão de variáveis categóricas e numéricas e pipeline de pré-processamento em Python, e todos esses elementos foram incorporados ao projeto.

O resumo da base indica que 30 registros duplicados foram removidos, reduzindo o conjunto de 869 para 839 linhas limpas. Essa decisão evita redundâncias desnecessárias e reduz o risco de o modelo aprender padrões artificiais a partir de repetições idênticas.

Na sequência, as colunas textuais foram padronizadas com uma função de normalização que converte os valores para minúsculas, remove espaços extras e corrige inconsistências simples identificadas na base. Essa etapa reduz fragmentação categórica e melhora a qualidade semântica das entradas.

Embora o arquivo `missing_values.csv` indique ausência de nulos explícitos após a limpeza, a criação das colunas derivadas `unique_symptom_count`, `unknown_symptom_count` e `symptom_text_length` permitiu monitorar, de forma indireta, a completude e o padrão dos sintomas em cada registro. Em especial, o resumo numérico mostra que o número de sintomas distintos por registro é muito estável, e que não há sintomas marcados como “unknown” na versão final da base.

A variável-alvo original `Dangerous` foi convertida para a variável binária `dangerous_binary`. Adicionalmente, foram criados atributos derivados (`symptoms_text`, `unique_symptom_count`, `unknown_symptom_count`, `symptom_text_length`) que sintetizam de maneira quantitativa o conteúdo das cinco colunas de sintomas.

Na transformação final para modelagem, foi utilizado `ColumnTransformer`, combinando `OneHotEncoder(handle_unknown="ignore")` para as variáveis categóricas e `StandardScaler` para as variáveis numéricas derivadas, tudo encapsulado em um `Pipeline` para reduzir o risco de vazamento de dados.

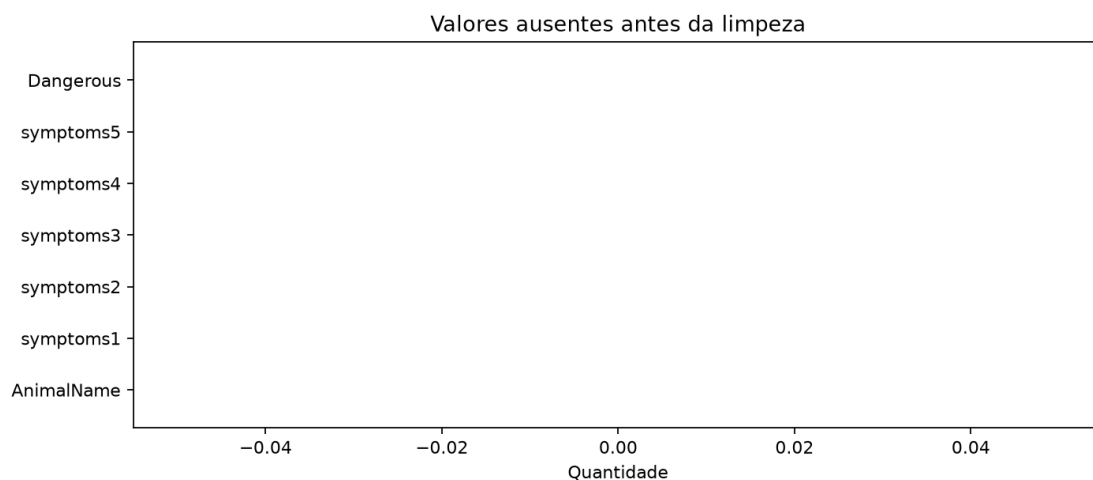


Figura 4 – Quantidade de valores ausentes por coluna antes da etapa de limpeza. Nesse caso, não existiam valores nulos ou ausentes pois já estamos trabalhando com o dados tratados.

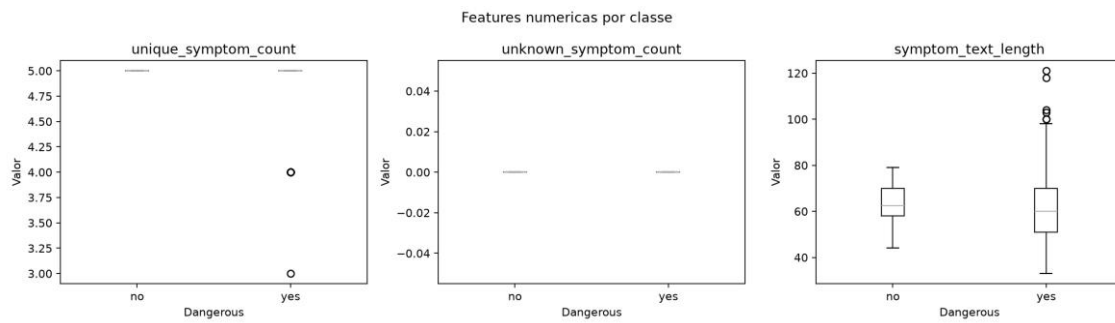


Figura 5 – Distribuição das features numéricas derivadas por classe da variável-alvo.

Comparação entre dataset original e dataset limpo

Durante o desenvolvimento, foram realizados testes tanto com o dataset original quanto com a versão já limpa e padronizada. A versão limpa, com duplicatas removidas e sintomas normalizados, mostrou-se mais coerente com a proposta de triagem, pois reduziu ruídos e inconsistências sem sacrificar variabilidade relevante.

Na prática, isso se refletiu em métricas mais estáveis entre treino, validação e teste, além de explicações de importância de atributos mais consistentes. Por esse motivo, optou-se por adotar o dataset já trabalhado como base principal para os experimentos, priorizando produtividade e assertividade na construção do modelo.

6. Análise de correlação

A análise de correlação foi realizada para identificar variáveis mais associadas à classe-alvo e apoiar a interpretação dos dados, em linha com o que o desafio solicita. No script, essa etapa combina medidas de associação entre as variáveis codificadas e a variável binária de saída, além de avaliar a relevância das features derivadas.

As importâncias calculadas via `permutation_importance` indicam que as colunas de sintomas (`symptoms1` a `symptoms5`) e o próprio `AnimalName` têm impacto relevante na predição, seguidas por `symptom_text_length`; já `unique_symptom_count` e `unknown_symptom_count` aparecem com contribuição nula nesse método. Essa leitura é coerente com a expectativa de que o conteúdo textual dos sintomas agregue mais sinal do que o número absoluto de sintomas registrados.

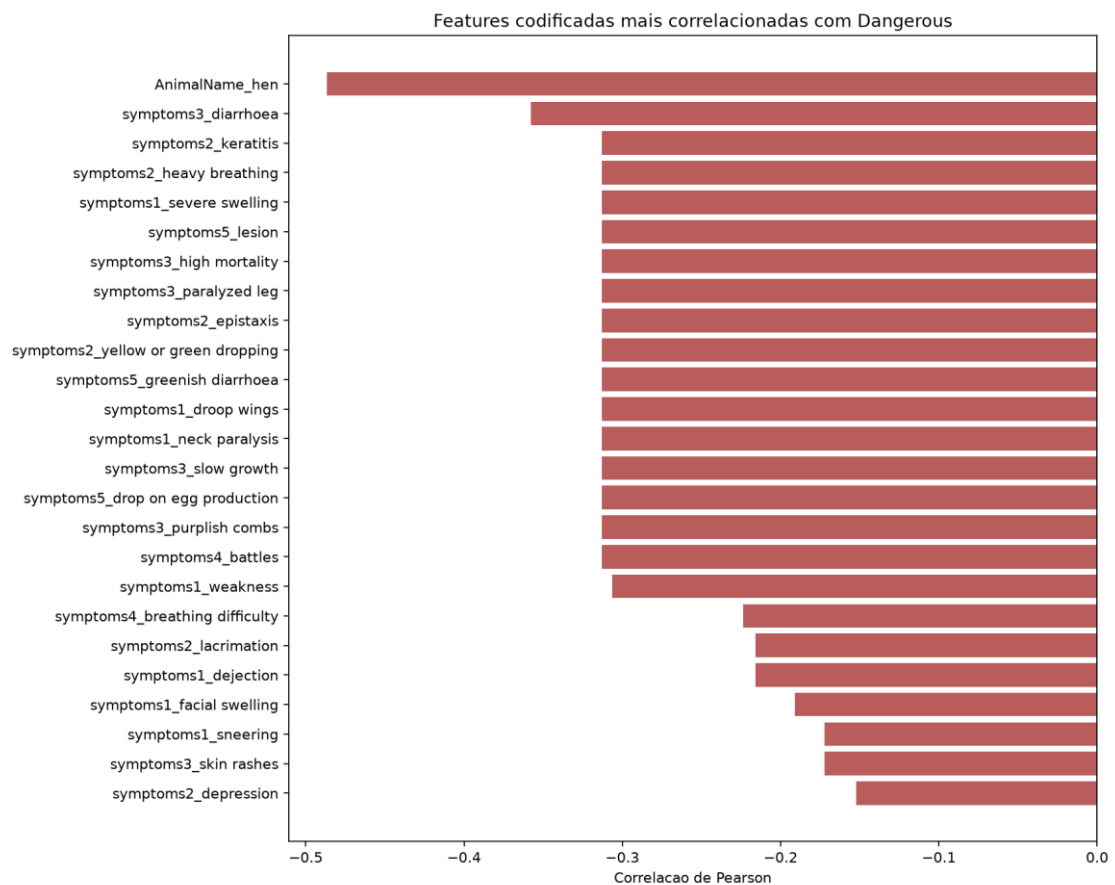


Figura 6 – Principais variáveis codificadas com maior correlação de Pearson com a variável-alvo binária.

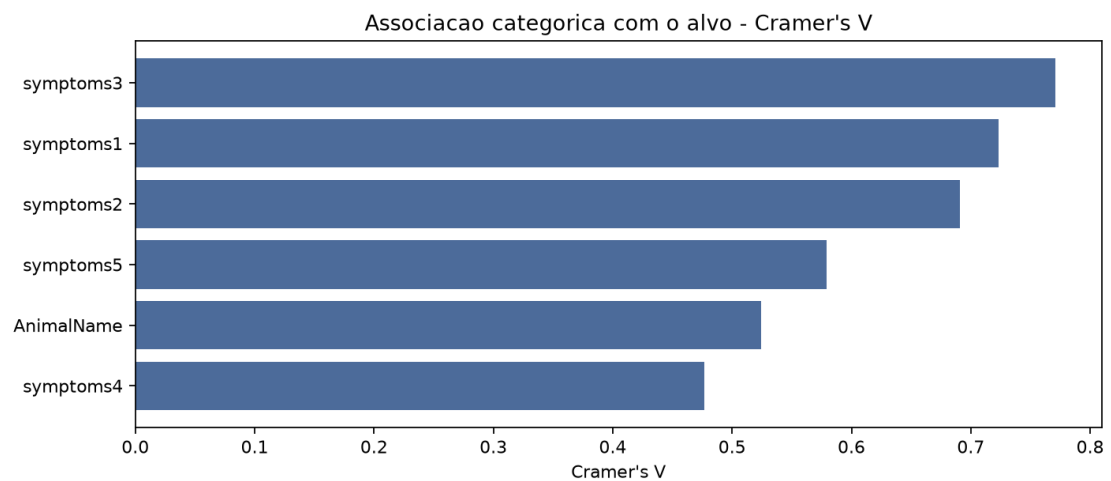


Figura 7 – Associação entre variáveis categóricas e a variável-alvo medida por Cramér's V.

7. Modelos utilizados e justificativa

Foram treinados quatro modelos de classificação supervisionada: Regressão Logística, Árvore de Decisão, Random Forest e K-Nearest Neighbors (KNN). Essa escolha atende ao requisito do desafio de utilizar duas ou mais técnicas e

permite comparar abordagens lineares, baseadas em regras, ensemble e vizinhança em um mesmo problema.

- Regressão Logística: escolhida por ser um modelo clássico de classificação binária, eficiente e relativamente interpretável; foi configurada com `class_weight="balanced"`, `solver="liblinear"` e `max_iter=2000`.
- Árvore de Decisão: permite capturar relações não lineares e gerar regras intuitivas de separação entre classes.
- Random Forest: faz ensemble de diversas árvores, reduzindo variância e melhorando generalização, especialmente útil com grande quantidade de variáveis one-hot.
- KNN: representa uma abordagem baseada em proximidade, classificando novas amostras conforme vizinhos mais próximos; foi configurado com `n_neighbors=7` e `weights="distance"`.

A comparação entre esses quatro modelos foi importante porque permitiu observar diferentes formas de aprendizado sobre o mesmo conjunto de dados e selecionar empiricamente a melhor solução em vez de assumir previamente qual algoritmo seria superior.

8. Separação entre treino, validação e teste

A divisão dos dados foi realizada de forma explícita em treino, validação e teste, conforme exigido no enunciado do desafio. O script utiliza `train_test_split` com estratificação pela variável-alvo, preservando a proporção entre as classes em cada subconjunto.

O arquivo `split_summary.json` documenta essa divisão: 503 linhas foram destinadas ao treino, 168 à validação e 168 ao teste. Também são listadas as colunas utilizadas como atributos (`AnimalName`, `symptoms1` a `symptoms5`, `unique_symptom_count`, `unknown_symptom_count`, `symptom_text_length`), bem como sua separação em variáveis categóricas e numéricas.

Essa separação é metodologicamente importante porque reduz o risco de superestimar o desempenho do sistema. O conjunto de treino é usado para ajuste dos modelos, o conjunto de validação para seleção da melhor alternativa e o conjunto de teste para medir a capacidade de generalização em dados não vistos durante a etapa de escolha.

9. Critério de escolha do melhor modelo

O melhor modelo foi selecionado com base principalmente no `recall_yes` (classe positiva), utilizando `f1_yes` e `accuracy` como critérios de desempate. Essa decisão foi tomada porque, em um contexto de triagem, é mais crítico deixar de identificar um caso perigoso do que gerar um alerta falso.

As métricas de validação mostram que Regressão Logística e Random Forest alcançaram accuracy de 0,9762, recall_yes em torno de 0,9939 e f1_yes próximo de 0,9879, com ROC AUC acima de 0,94. O KNN apresentou recall máximo (1,0), porém com ROC AUC consideravelmente menor, enquanto a Árvore de Decisão ficou abaixo dos demais em acurácia e F1.

Diante desse cenário, a equipe priorizou o modelo que equilibrou alto recall com qualidade global da curva ROC, favorecendo as soluções com melhor ROC AUC e F1 em vez de olhar apenas a métrica de sensibilidade isolada.

10. Treinamento e avaliação

Todos os modelos foram treinados sobre o conjunto de treino e avaliados no conjunto de validação utilizando métricas padronizadas. O script calcula accuracy, precision_yes, recall_yes, f1_yes, f1_weighted e roc_auc, além de gerar classification_report e matrizes de confusão para diferentes subconjuntos.

Após a comparação inicial entre os classificadores, o melhor modelo é selecionado e submetido a uma avaliação mais detalhada nos conjuntos de treino, validação e teste. Esse procedimento permite verificar consistência de desempenho e observar possíveis indícios de sobreajuste ou perda de generalização.

As matrizes de confusão documentadas permitem analisar explicitamente a quantidade de verdadeiros positivos, verdadeiros negativos, falsos positivos e falsos negativos nos conjuntos de validação e teste. Em um sistema de apoio à triagem, os falsos negativos merecem atenção especial, pois representam situações em que o sistema deixa de sinalizar um registro potencialmente perigoso.

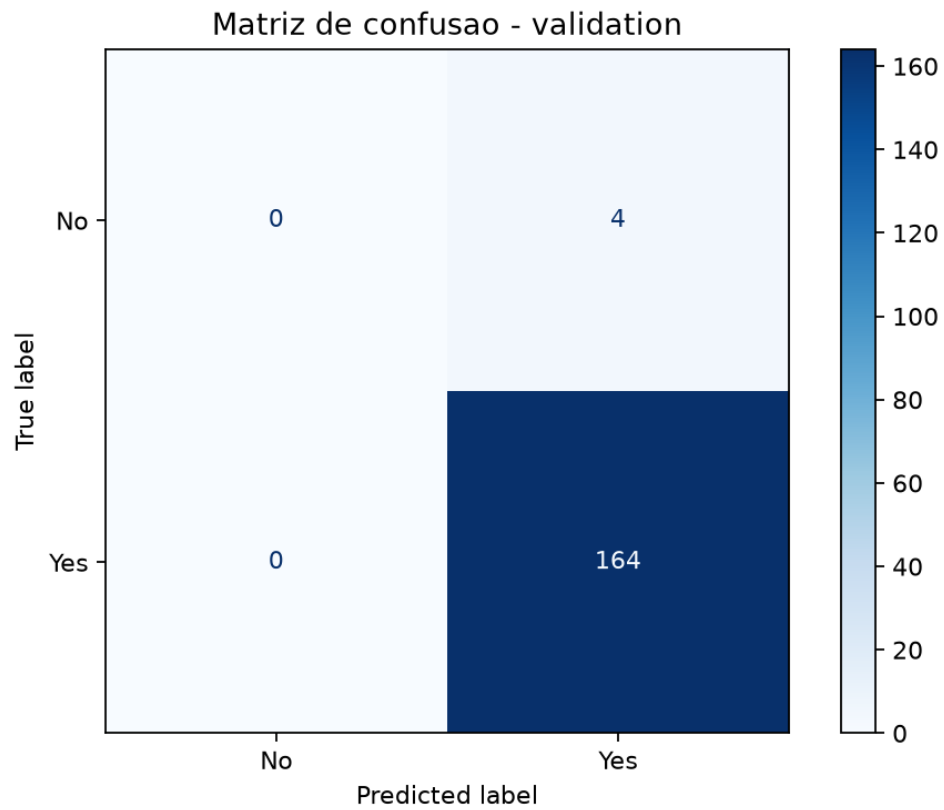


Figura 8 – Matriz de confusão do modelo selecionado no conjunto de validação.

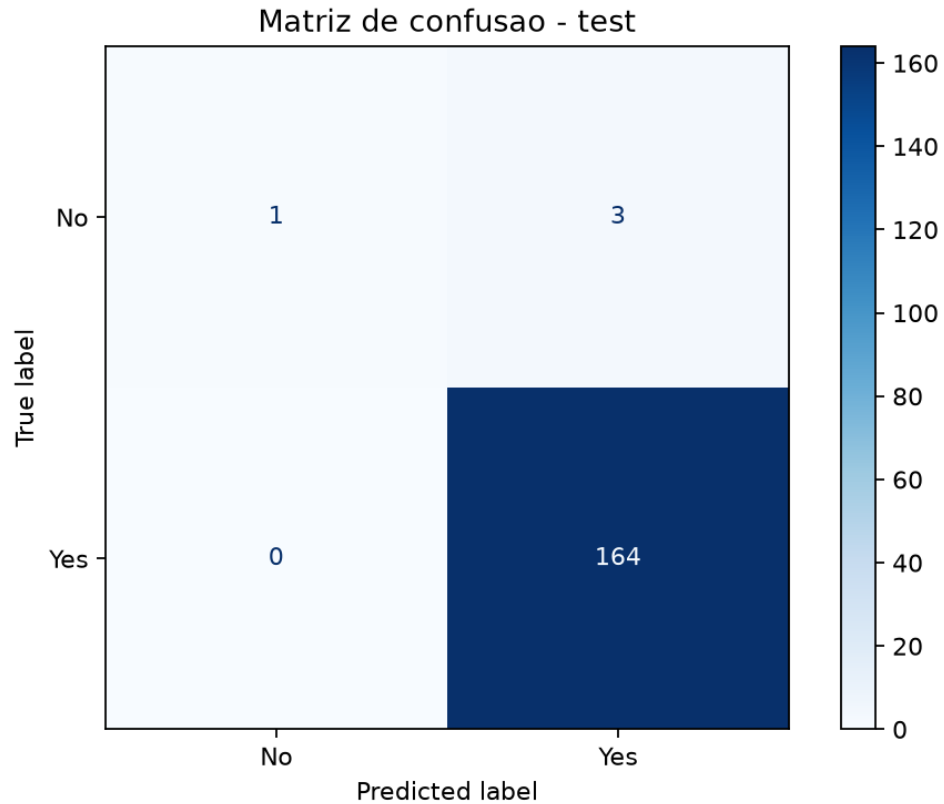


Figura 9 – Matriz de confusão do modelo selecionado no conjunto de teste.

11. Resultados e interpretação dos dados

A análise dos resultados considera tanto as métricas quantitativas quanto a interpretabilidade do modelo. Em um problema de triagem, não basta observar quantos casos foram classificados corretamente em termos gerais; é necessário avaliar se o sistema consegue identificar de maneira satisfatória os casos mais críticos.

Os resultados de validação sinalizam que é possível alcançar acurácia alta (em torno de 0,9762), recall elevado para a classe perigosa (até 0,9939) e F1-score superior a 0,98 com as melhores configurações testadas. O ROC AUC próximo de 0,96 nos melhores modelos também indica boa separação global entre as duas classes, mesmo em um cenário de forte desbalanceamento.

Além das métricas, o pipeline também gera mecanismos de explicabilidade. O arquivo `feature_importance.csv` mostra as principais variáveis utilizadas, com destaque para as colunas de sintomas e para o tipo de animal como atributos mais relevantes, seguidas pelo comprimento do texto de sintomas. Essa informação ajuda a entender por que determinadas previsões são favorecidas e reforça a centralidade dos sintomas na decisão do modelo.

O arquivo `prediction_example.json` exemplifica a saída do modelo em um caso concreto, no qual um registro de bird com sintomas graves é classificado como “Yes - perigoso” com probabilidade 1,0, acompanhado de um aviso explícito de que a previsão é apenas apoio à triagem e que a decisão final deve ser do profissional de saúde.

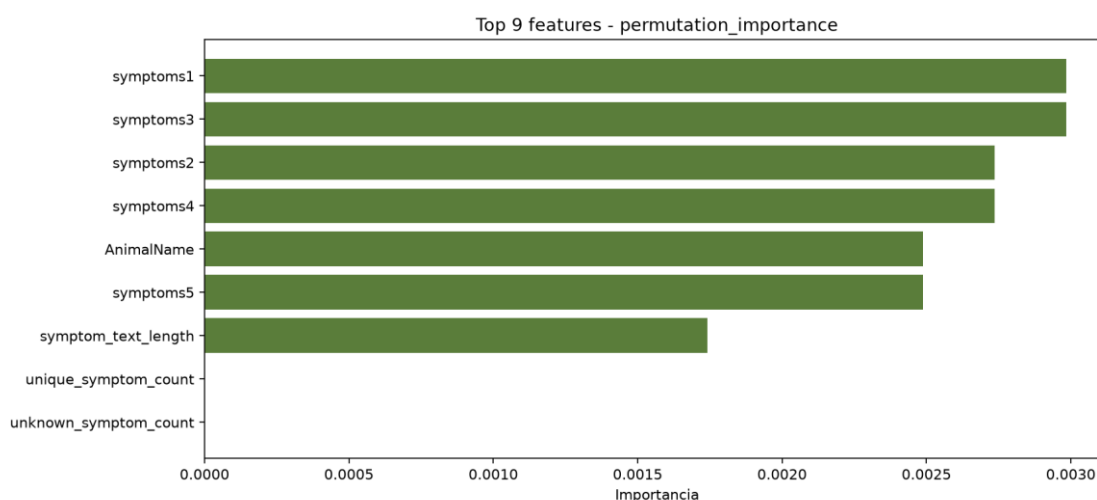


Figura 10 – Importância das principais variáveis utilizadas pelo modelo na classificação dos registros.

12. Discussão crítica dos resultados

Embora o projeto apresente um pipeline completo e metodologicamente adequado, é importante reconhecer suas limitações. O modelo foi treinado em uma base específica, com variáveis categóricas simples e um contexto delimitado, o que significa que sua capacidade de generalização para outros cenários precisa ser validada com cautela.

O desbalanceamento da variável-alvo, com 819 casos yes e apenas 20 no, sugere que métricas como acurácia devem ser interpretadas com cuidado, mesmo que o modelo tenha apresentado recall elevado para a classe positiva. Em contextos clínicos reais, seria recomendável complementar essa avaliação com validação externa e análise de sensibilidade a diferentes distribuições de classes.

Outro ponto relevante é que correlação, importância de atributos e explicabilidade estatística não devem ser confundidas com causalidade. Uma variável pode aparecer como muito relevante para a previsão no conjunto estudado sem que isso signifique uma relação causal robusta em um contexto clínico mais amplo.

Também é necessário destacar que a base utilizada simplifica a realidade ao trabalhar com poucos atributos textuais curtos e não contemplar outros tipos de informação que poderiam ser decisivos, como exames laboratoriais, histórico clínico e dados temporais. Por isso, a aplicação prática do modelo exige cuidado e não pode ser tratada como substituição da análise especializada.

13. Possibilidade de uso prático

Na prática, uma solução como esta pode ser utilizada como ferramenta de apoio à triagem, ajudando a destacar casos com maior probabilidade de risco e contribuindo para a priorização inicial do atendimento. Em cenários com grande volume de registros, isso pode trazer ganhos de organização e eficiência no fluxo de análise.

No entanto, qualquer uso real exigiria validação externa, revisão especializada, monitoramento contínuo, análise de possíveis vieses e integração responsável aos processos de decisão. O modelo deve ser entendido como mecanismo de suporte e não como substituto da decisão humana.

14. Conclusão

Este trabalho apresentou o desenvolvimento de um pipeline completo de Machine Learning para classificação binária em dados tabulares, atendendo aos principais requisitos técnicos do Tech Challenge da Fase 1. Foram contempladas as etapas de exploração dos dados, pré-processamento, análise de correlação, treinamento de múltiplos modelos, avaliação por métricas adequadas, interpretação dos resultados e organização dos artefatos necessários para entrega.

Ao longo do desenvolvimento, foram testadas tanto a versão original quanto a versão limpa do dataset, e observou-se que a base trabalhada, com duplicatas removidas e sintomas padronizados, proporcionou métricas mais consistentes e uma leitura mais clara das importâncias de atributos. Por isso, essa versão foi adotada como referência para o modelo final, priorizando produtividade e assertividade na construção da solução.

A solução proposta mostrou-se adequada como prova de conceito para apoio à triagem, especialmente porque a escolha do modelo priorizou o recall da classe

perigosa, em alinhamento com o risco associado ao problema. Ainda assim, a utilização prática do sistema deve ocorrer com cautela, sempre com supervisão profissional e validação adicional antes de qualquer aplicação em ambiente real.

15. Link do repositório Git

https://github.com/RamonCintas/Smart_VET.git

FIAP – Pós-Tech

Tech Challenge – Fase 1

Atividade extra – Visão Computacional com Imagens

Relatório Técnico – Sistema de Visão Computacional para Identificação de Raças e Triagem Dermatológica em Cães

Alunos:

Eduardo Coimbra Mattos
Guilherme dos Santos Condini
Paulo Henrique de Castilho
Ramon Cintas Gomes

Curso: IA para DEVs

SUMÁRIO

1. Introdução
2. Objetivo
3. Bases de dados utilizadas
4. Organização e engenharia dos dados
5. Pré-processamento e pipeline de entrada
6. Arquitetura dos modelos
7. Estratégia de treinamento
8. Resultados obtidos
 - 9.1 Tabela de métricas
9. Inferência interativa e interpretabilidade
10. Discussão crítica
11. Potencial de aplicação prática
12. Conclusão
13. Itens recomendados para anexar na versão final em PDF

1. Introdução

Este relatório técnico apresenta o desenvolvimento de uma solução de visão computacional voltada à análise automatizada de imagens de cães, com duas frentes complementares: identificação de raças e triagem dermatológica assistida por inteligência artificial. O projeto foi desenvolvido como atividade extra da Fase 1 do Tech Challenge, cuja proposta prevê explicitamente o uso opcional de redes neurais convolucionais para diagnóstico por imagens como complemento à entrega principal baseada em dados estruturados.

A proposta parte da premissa de que sistemas baseados em aprendizado profundo podem atuar como ferramentas de apoio à triagem inicial, contribuindo para acelerar a análise de imagens e destacar indícios relevantes para avaliação humana. Ainda que a decisão final deva permanecer com o profissional responsável, modelos desse tipo podem gerar valor em etapas preliminares de classificação, priorização e apoio à interpretação visual.

Neste trabalho, a solução foi construída para combinar duas tarefas em um único fluxo: sugerir a raça mais provável do animal e indicar possíveis condições dermatológicas visualmente detectáveis. Essa abordagem tornou o projeto mais completo do ponto de vista técnico e mais coerente com a ideia de suporte inteligente à análise de imagens.

2. Objetivo

O objetivo do projeto foi desenvolver um sistema capaz de analisar imagens de cães e retornar, de forma automatizada, as classes mais prováveis associadas tanto à raça do animal quanto a possíveis doenças de pele observáveis visualmente. A proposta buscou demonstrar a aplicação prática de redes neurais convolucionais em um cenário de triagem assistida por imagens.

De maneira mais específica, o trabalho teve como metas:

- Consolidar diferentes conjuntos de imagens em uma base única para treinamento e validação.
- Aplicar transferência de aprendizado com arquiteturas modernas de classificação visual.
- Melhorar a capacidade de generalização por meio de pré-processamento e aumento de dados.
- Disponibilizar uma rotina interativa de inferência com saída interpretável para demonstração do sistema.

3. Bases de dados utilizadas

O notebook utilizado como base principal (Dog_raca_e_doenca.ipynb) integra múltiplas fontes de imagens em um único pipeline. Durante a etapa de carga, o sistema registra o uso do workspace da Roboflow, da cache do Colab para o dataset dogs-skin-diseases-image-dataset e do mapeamento estrutural dos arquivos antes da criação dos conjuntos de treino e validação.

Após essa consolidação, a base final ficou composta por 21.299 imagens distribuídas em 137 classes. Desse total, 17.040 imagens foram direcionadas ao conjunto de treinamento e 4.259 ao conjunto de validação, mantendo uma separação proporcional de 80% para treino e 20% para validação.

Essa organização permitiu reunir, em um mesmo espaço de classes, categorias ligadas a raças caninas e categorias relacionadas a doenças dermatológicas. Na prática, isso viabilizou um fluxo de inferência único, no qual a imagem enviada pelo usuário pode ser analisada sob mais de uma perspectiva sem necessidade de múltiplos sistemas independentes.

4. Organização e engenharia dos dados

Uma etapa importante do projeto foi a preparação estrutural da base para leitura pelo TensorFlow. O notebook mostra a execução de uma fase de “mapeamento de arquivos estruturais”, indicando que os dados foram organizados antes da montagem do pipeline de entrada, com padronização suficiente para uso direto por rotinas de classificação supervisionada.

Em seguida, a solução utiliza a função `image_dataset_from_directory`, que exige uma organização consistente por classes em diretórios separados. Isso indica que o projeto foi montado com uma estrutura clara de rotulagem, o que é essencial em tarefas com grande número de classes e fontes distintas de imagem.

Do ponto de vista metodológico, essa etapa é fundamental porque a qualidade da engenharia dos dados influencia diretamente o desempenho do modelo. Em problemas de visão computacional, inconsistências de rotulagem, diretórios mal organizados ou distribuição desequilibrada entre classes podem comprometer significativamente a generalização da rede neural.

5. Pré-processamento e pipeline de entrada

O pipeline de entrada foi construído com `tf.keras.utils.image_dataset_from_directory`, utilizando `validation_split=0.2`, `seed=42`, imagens redimensionadas para 160x160 pixels e batch size igual a 64. Esses parâmetros definem uma rotina padronizada de leitura e particionamento dos dados, com separação clara entre treino e validação.

Após a leitura, os conjuntos foram otimizados com operações como `cache()`, `shuffle()` e `prefetch()` com AUTOTUNE, o que contribui para reduzir gargalos de leitura e melhorar o aproveitamento computacional durante o treinamento em GPU. Esse cuidado é importante em bases extensas, como a utilizada neste projeto, que reúne mais de 21 mil imagens.

Também foram aplicadas técnicas de aumento de dados com o objetivo de aumentar a robustez do modelo. O notebook utiliza transformações aleatórias típicas para visão computacional, como inversão horizontal, rotação, zoom e ajuste de contraste, permitindo maior variabilidade visual no conjunto de treino.

Esse conjunto de técnicas ajuda a reduzir overfitting e melhora a capacidade de generalização do modelo, especialmente em problemas nos quais mudanças de iluminação, enquadramento e textura da pele podem influenciar a classificação. Em uma tarefa que combina raça e condição dermatológica, esse ponto se torna ainda mais relevante.

6. Arquitetura dos modelos

A arquitetura principal escolhida para o sistema foi a EfficientNetV2-B0, empregada com pesos pré-treinados e adaptada ao problema por meio de uma nova cabeça classificadora. Sobre a base convolucional foram adicionadas camadas de pooling global, Dropout(0.2), uma camada densa intermediária e uma camada final softmax, compatível com o problema multiclasse proposto.

A escolha da EfficientNetV2-B0 foi adequada porque essa arquitetura oferece bom equilíbrio entre desempenho e custo computacional. Em cenários acadêmicos com limitação de recursos, ela permite obter resultados expressivos em classificação de imagens sem exigir estruturas extremamente pesadas.

Além do modelo principal, o sistema também utiliza uma MobileNetV2 auxiliar como mecanismo de filtragem. Essa rede secundária atua como detector de felinos, impedindo que o classificador principal de cães seja aplicado sobre imagens fora do escopo pretendido, o que ajuda a reduzir inferências inadequadas.

7. Estratégia de treinamento

O treinamento foi dividido em duas fases, o que representa uma estratégia clássica de transferência de aprendizado. Na Fase 1, descrita no notebook como “Aquecimento da Cabeça”, a base convolucional foi mantida congelada e apenas as camadas finais foram ajustadas ao novo problema ao longo de 8 épocas.

Na Fase 2, o notebook registra a ativação do fine-tuning das camadas superiores. Nessa etapa, parte da rede pré-treinada foi descongelada para refinar as representações profundas, permitindo melhor adaptação ao domínio específico das imagens caninas e dermatológicas.

Essa estratégia faz sentido porque reduz o risco de destruir o conhecimento previamente aprendido em ImageNet logo no início do treinamento. Primeiro a cabeça classificadora se adapta ao problema; depois, as camadas profundas são refinadas de maneira mais controlada.

8. Resultados obtidos

Na Fase 1, o modelo apresentou melhora consistente ao longo do treinamento. A val_accuracy evoluiu de 0,7560 na primeira época para 0,8162 na oitava época, enquanto a val_loss caiu de 0,8015 para 0,5953, indicando aprendizado progressivo e estabilidade do processo.

Na Fase 2, após a ativação do fine-tuning, a `val_accuracy` partiu de 0,7215 e chegou a 0,7960 na sexta época, ao passo que a `val_loss` reduziu de 0,9567 para 0,6678. Embora os resultados finais da segunda fase tenham ficado ligeiramente abaixo do melhor ponto da fase inicial, houve avanço consistente ao longo das épocas exibidas.

Em termos gerais, os resultados mostram que o modelo foi capaz de aprender representações úteis mesmo diante de uma base ampla, com 137 classes e diferentes tipos de padrão visual. Isso sugere que a combinação de arquitetura eficiente, augmentation e transferência de aprendizado foi adequada ao objetivo do projeto.

8.1 Tabela de métricas

Etapa	Métricas principais
Fase 1 – Aquecimento da cabeça	<code>val_accuracy</code> de 0,7560 para 0,8162; <code>val_loss</code> de 0,8015 para 0,5953
Fase 2 – Fine-tuning	<code>val_accuracy</code> de 0,7215 para 0,7960; <code>val_loss</code> de 0,9567 para 0,6678
Base consolidada	21.299 imagens, 137 classes; 17.040 treino e 4.259 validação

9. Inferência interativa e interpretabilidade

Um diferencial importante da solução foi a criação de uma rotina interativa de inferência. Ao final do treinamento, o notebook solicita o envio da foto do animal e, a partir dessa entrada, executa o fluxo de diagnóstico assistido pelo modelo.

A proposta do sistema é apresentar resultados de forma interpretável, com apoio visual e probabilístico. Em vez de limitar a saída a uma única classe, a rotina foi pensada para exibir predições ordenadas por confiança e apoiar a leitura humana do resultado, o que é mais útil em um contexto de triagem e demonstração acadêmica.

Essa característica aproxima o projeto de uma aplicação prática de apoio à decisão, pois sistemas de triagem tendem a ser mais úteis quando mostram níveis de confiança, hipóteses concorrentes e alertas interpretáveis. Em um cenário real, esse formato permitiria ao usuário analisar a resposta do modelo com maior senso crítico.

10. Discussão crítica

Os resultados obtidos mostram que a solução é tecnicamente consistente como protótipo acadêmico de visão computacional. A combinação de EfficientNetV2-B0, aumento de dados, pipeline otimizado e treinamento em

duas fases produziu níveis sólidos de desempenho para um problema multiclasse com grande volume de imagens.

Apesar disso, algumas limitações devem ser destacadas. A base consolidada reúne classes de natureza distinta, incluindo raças e condições dermatológicas, o que aumenta a complexidade do problema e pode introduzir vieses de distribuição. Além disso, o notebook apresenta treino e validação, mas não evidencia de forma explícita um conjunto de teste independente separado especificamente para avaliação final.

Outro ponto central é que o sistema não realiza diagnóstico veterinário definitivo. Seu papel é o de apoio à triagem inicial por imagem, oferecendo probabilidades e indícios automatizados que podem auxiliar a análise humana. Essa interpretação é coerente com o enunciado do desafio, que ressalta que a palavra final no diagnóstico deve sempre permanecer com o profissional especializado.

11. Potencial de aplicação prática

Em um cenário real, a solução poderia ser aplicada em clínicas veterinárias, plataformas de atendimento remoto, sistemas de triagem preliminar ou contextos educacionais. Nesses ambientes, o modelo poderia ajudar a sinalizar imagens com maior probabilidade de alteração dermatológica e organizar a priorização para análise humana.

Também há forte valor didático no projeto, pois ele demonstra um pipeline completo de visão computacional: organização das bases, pré-processamento, modelagem, treinamento, validação e inferência interativa. Para fins acadêmicos, isso evidencia domínio dos fundamentos de deep learning aplicado a imagens.

12. Conclusão

Este projeto apresentou o desenvolvimento de um sistema de visão computacional para identificação de raças e triagem dermatológica em cães, utilizando redes neurais convolucionais e transferência de aprendizado. A solução foi estruturada sobre a EfficientNetV2-B0 como modelo principal, com MobileNetV2 como filtro auxiliar e um pipeline de entrada otimizado para treinamento em GPU.

Os resultados mostraram desempenho consistente ao longo das duas fases de treinamento, com melhor val_accuracy de 0,8162 na etapa de aquecimento da cabeça e progressão contínua durante o fine-tuning. A base consolidada, com 21.299 imagens e 137 classes, mostrou-se suficiente para validar a proposta de um protótipo funcional de classificação visual aplicada à triagem assistida.

Como atividade extra da Fase 1, o projeto atende adequadamente à proposta de aplicação de visão computacional prevista no desafio. Ainda que sua adoção prática exija validações adicionais, refinamento metodológico e maior cuidado com avaliação clínica, o trabalho demonstra consistência técnica,

organização e potencial de evolução para cenários mais robustos de apoio à decisão por imagens.

13. Link do Github

https://github.com/RamonCintas/Smart_VET.git